How to Use Your Website

A practical guide for editing and maintaining your personal website

Ana Sokolova

Personal Academic Website

Note

This guide walks you through everything you need to run and edit the website yourself: installing the tools, understanding the folder structure, changing text and images, previewing your work, and publishing it online. It assumes you are comfortable reading and writing HTML, but does *not* assume any deeper programming experience. Anything more complex will be handled by me.

Website design and development by Samuel Lackner — samuel-lackner.at

Contents

1. Overview: How the Website Works

Before diving in, here is the big picture in plain terms.

Your website is a **static site**: it is a collection of plain HTML, CSS, and JavaScript files. There is no database and no server-side software to worry about. To avoid repeating the same header, footer, and sidebar on every page, the site is assembled by a small **build tool** called Eleventy (11ty). You edit simple source files in the `src/` folder; the build tool stitches them together into the finished website.

```
You edit files in src/   ⟶  the build tool assembles them   ⟶
finished site appears in _site/   ⟶  GitHub publishes it online
```

The hosting and rebuilding happen automatically on **GitHub**: every time you save your changes to the repository, the site is rebuilt and published for you. You never have to upload files by hand.

Tip

You only ever need to edit files inside the `src/` folder. Everything outside it is either configuration (handled by me) or output generated automatically.

2. Setup and Installation

You only need to do this section *once*, when you first set up the project on your computer.

2.1 What you need

- A **terminal** (the "Command Prompt" or "Terminal" application on your computer).
- **Node.js**, which provides the `npm` command used to install and run the build tool.

Node.js can be downloaded and installed from:

https://nodejs.org/en/download

Installing Node.js automatically includes `npm`, so you do not need to install anything else separately. You will need it for the `npm install` and `npm run serve` commands later.

Tip

To check that everything installed correctly, open a terminal and type the two commands below. If each prints a version number, you are ready.

```
node -v
npm -v
```

2.2 Downloading the project

Download ("clone") the project from GitHub by running this command in your terminal:

```
git clone https://github.com/FungalCode/sokolova-ana.git
```

This creates a folder containing all the website files. You can then transfer all the files into your *own* GitHub repository, so that you are the owner of the published site.

2.3 Enabling GitHub Pages

The website is hosted for free using **GitHub Pages**. In your new repository this must be switched on *once*, and set to the correct mode.

Important

In your GitHub repository, go to:

Settings → Pages → Source

and choose **"GitHub Actions"** — *not* "Deploy from a branch".

This is essential. The site is built by an automated GitHub Actions workflow, and that only works when this option is selected. With the wrong setting, publishing will fail.

2.4 How building and publishing works

Because the site is built by GitHub Actions, it **automatically rebuilds every time you push changes** to the repository. There is nothing to upload manually — you save your changes, and a minute or so later the live site is updated.

Note

First-time exception. Right after setup, if you have not pushed any changes yet, the site will not have been built. You have two ways to trigger the very first build:

1. Make any small change and push it, which starts the build automatically; *or*
2. Trigger the build by hand: in the GitHub repository, go to **Actions → Deploy to GitHub Pages** (menu on the left) → **Run workflow → Run workflow**.

2.5 Getting later updates

If I make improvements to the project and you want to pull them into your copy, run this inside the project folder:

```
git pull origin main
```

Alternatively, you can simply clone the project again from scratch.

3. The Project Structure

Everything you will ever edit lives inside the `src/` folder.

Tip

You can safely ignore everything *outside* the `src/` folder. Those files are configuration and build machinery.

3.1 The page files

Inside `src/` you will find five HTML files, one for each page of the website:

`index.html`	Home
`about.html`	Personal / About
`papers.html`	Papers
`talks.html`	Talks
`teaching.html`	Teaching

There is also a file called `src.11tydata.js`. This is part of the build tool and can be ignored — it simply connects each page to its web address (for example, it makes `www.example.com/papers.html` come from `papers.html`).

3.2 `_includes` — the reused pieces

This folder holds all the **recurring parts** of the website, so you only have to edit them in one place:

- `footer.njk` — the footer at the bottom of every page.
- `header.njk` — the header / navigation bar at the top of every page.
- `sidebar.njk` — the sidebar on the subpages, showing your image, title, name, contact details, and links.
- `base.njk` — combines everything above into a complete, working HTML page.

Note

The `.njk` files are "Nunjucks" templates. In practice they are mostly ordinary HTML, with a little extra template syntax. As long as you keep to the existing layout, editing them is just like editing HTML.

3.3 `assets` — images, icons, fonts, and papers

This folder contains all images, icons, fonts (in the `fonts` subfolder), and PDF papers (in the `papers` subfolder). Put any new images and papers here, and make sure to reference them correctly, for example:

```
assets/papers/doc.pdf
assets/image.png
```

Important

The path always begins with `assets/`. If you write the path without it, the image or PDF will not be found and will not appear on the site.

3.4 `js` — the JavaScript

There are only two small JavaScript files, and you will rarely need to touch them:

- `nav.js` — makes the "burger" menu work on mobile screens.
- `toc.js` — makes the table of contents on the homepage work (the one on the left, next to the content).

3.5 `css` — the styling

Edit these files whenever you want to change how things *look* — text size, spacing between elements, colors, and so on.

- `base.css` — defines the variables for all colors and the like (for example `--text-l0: #050505;`), plus basics such as the font and the buttons.
- `layout.css` — styling for the body, header, container, and footer.
- `content.css` — everything for the content: the text, sidebars, images, and links (for example, the link to Google Scholar).
- `hero.css` — specifics for the first section of the homepage: the contact grid, the "calm" logo, the image, and so on.
- `fonts.css` — a simple file that loads the fonts.

4. Making Changes

4.1 Previewing your changes locally

Before publishing anything, you can preview your changes on your own computer. In the project folder, run:

```
npm run serve
```

Note

The very first time only, you must install the tools once beforehand:

```
npm install
```

You only ever need to do this once per computer.

`npm run serve` starts a small web server on your machine and prints a link, usually:

http://localhost:8080

Open that link in your browser to see the site exactly as it will appear online. It updates automatically as you edit and save. When you are finished previewing, you can stop the server by pressing `Ctrl + C` in the terminal.

4.2 Editing the meta description and title

The **title** and **description** are what appear on a Google search results page and in the browser tab. Each page has these at the very top, written between two lines of three dashes (`---`):

```
---
layout: base.njk
title: About - Ana Sokolova
extraCss:
  - css/content.css
---
```

You can change the title, and optionally add a description:

```
---
layout: base.njk
title: New Title
description: New description
extraCss:
  - css/content.css
---
```

Important

Do *not* change the `layout`, `extraCss`, or `extraJs` lines. These tell the build tool how to assemble the page and which stylesheets and scripts it needs. Only edit the `title` and `description`.

4.3 Editing the page content (HTML)

The homepage is separated into two parts: the **hero section** at the top and the **content section** below it.

In the content section you can freely type any HTML — headings, links, tables — and it will always look fine. The subpages work the same way, and are simpler.

- **Main content on the homepage:** open `index.html` and scroll down to `<main class="content">`. Inside it you may freely write any valid HTML: different headings (`h1`, `h2`, `h3`, ...), links, tables, and so on.
- **Main content on a subpage:** the same, but simpler — you will find `<main class="content">` right at the top of the file.

Tip

The hero section is more delicate. It is a more complex layout, so take a little care there. You can still change any text — such as your phone number or email — but if something becomes much longer or shorter (for example a very long email address), just check afterwards that everything still looks good, including on your phone, in case something overflows. In practice this is rarely a problem.

4.4 Editing the recurring content (footer, header, sidebar)

To change the footer, header, or sidebar, edit the corresponding file in the `_includes` folder. This is mostly HTML with a little build-tool syntax. As long as you stick to the current layout, this should present no problems.

4.5 Changing the styling

All styling lives in the `css` folder. The site is styled with CSS *classes* (names beginning with a dot, such as `.content` or `.hero`) and, for colors and spacing, a set of *variables* defined once at the top of `base.css`. To change how something looks, find the matching class or variable in the file listed below and edit it.

Tip

The quickest way to find what controls an element is to open the site in your browser, right-click the element, and choose **"Inspect"**. The panel shows the exact class name (for example `.contact-cell`). Search for that name in the relevant CSS file to jump straight to it.

Global colors and spacing — top of `base.css`

These variables are defined once and reused everywhere, so changing one here updates the whole site at once. They sit in the `:root { ...}` block at the very top of `base.css`.

Variable	Controls
`--bg`	Page background color
`--text-l0/l1/l2`	Main text colors, darkest to slightly lighter
`--text-muted`	Muted grey text (footer, sidebar subtitles)
`--primary`	Accent color for content links and hovers (currently green)
`--cv-color`	Color of the "CV" link in the sidebar
`--border`	Color of card borders and dividing lines
`--container-padding`	Left/right page margin (space to the screen edge)
`--radius-card`, `--radius-button`, `--radius-anchor`	Corner roundness of cards, buttons, and icon tiles
`--icons-gap`	Spacing between the social-icon tiles
`--header-offset`	Height reserved for the sticky top header

Everything else — by file and class

For anything that is not a global color or spacing value, find the class below in the named file and edit its properties.

`base.css` — fonts and buttons

To change...	Look for...
The body font of the whole site	`body { font-family }`
Button look (Publications, Contact, etc.)	`.btn`, `.btn-primary`, `.btn-secondary`

`layout.css` — header, navigation, footer

To change...	Look for...
Overall page width and side padding	`.container`
The top navigation bar	`.site-header-inner`
The navigation links and their underline hover	`.site-header .nav-list a`
The mobile "burger" menu icon	`.burger`
The footer and its links	`.site-footer-inner`, `.footer-links`

`content.css` — page text, sidebar, links, table of contents

To change...	Look for...
Body paragraph text (size, color, spacing)	`.content p`
Headings within a page	`.content h1`, `.content h2`, `.content h3`
Links inside the content (e.g. Google Scholar)	`.content a`
Bulleted lists	`.content ul`
The Papers / Talks list rows	`.papers-list`, `.talks-list`
The two-column layout (sidebar + text)	`.content-section`
The subpage sidebar card (photo, name, contact)	`.subpage-sidebar`, `.sidebar-title`, `.sidebar-contact`
The social-icon tiles (Scholar, ORCID, CV...)	`.social-link`, `.socials`, `.associations`
The table of contents (homepage)	`.toc`, `.toc-list`
The photo on the About page	`.about-image`

`hero.css` — the homepage's first (top) section

To change...	Look for...
The overall hero layout	`.hero`
The name heading	`.hero-text h1`
The subtitle ("Full Professor")	`.title`
The contact grid (phone, mail, location, room)	`.contact-grid`, `.contact-cell`
The row of buttons	`.hero-actions`
The photo card	`.image-card`, `.hero-image`
The "calm" logo	`.calm_logo`

`fonts.css` — font loading

To change...	Look for...
Which font files are loaded	the `@font-face` blocks

Note

To switch to a different font entirely, you first add its `@font-face` import in `fonts.css` and then set the body's `font-family` in `base.css`.

5. Publishing Your Changes

When you are happy with your changes, publish them to the live website using the usual Git commands in your terminal, inside the project folder:

```
git add .
git commit -m "Describe what you changed"
git push
```

That is all. Pushing to the repository automatically triggers a rebuild on GitHub, and a minute or so later the live site reflects your changes.

6. Quick Reference

6.1 Commands cheat sheet

Command	What it does
`npm install`	Install the tools (once per computer)
`npm run serve`	Start a local preview at localhost:8080
`git pull origin main`	Get the latest updates from GitHub
`git add .`	Stage all your changes
`git commit -m "..."`	Save your changes with a message
`git push`	Publish your changes to the live site

6.2 Where do I edit. . . ?

I want to change. . .	Go to. . .
Homepage text	`src/index.html`
A subpage's text	`src/about.html`, `papers.html`, . . .
Header / navigation	`src/_includes/header.njk`
Footer	`src/_includes/footer.njk`
Sidebar	`src/_includes/sidebar.njk`
An image or PDF	add it to `src/assets/`
colors	top of `src/css/base.css`
Page title / description	top of the page's HTML file

7. Troubleshooting

The site did not update after I pushed.
Give it a minute or two — the rebuild takes a moment. You can watch its progress in the **Actions** tab on GitHub. If it never builds, check that GitHub Pages is set to **"GitHub Actions"** (see Section 2.3).

My image or PDF is not showing.
Check the path. It must begin with `assets/`, for example `assets/image.png`, and the file must actually be inside `src/assets/`.

`npm run serve` gives an error the first time.
Make sure you ran `npm install` once beforehand, and that Node.js is installed (`node -v` should print a version).

The very first build never ran.
Right after setup there are no changes to trigger it. Either push a small change, or trigger it by hand under **Actions** → **Deploy to GitHub Pages** → **Run workflow**.

Something looks broken and I cannot tell why.
This is exactly the kind of thing I will handle — get in touch and describe what you changed.

For anything more complex than the above, get in touch with me, Samuel Lackner.